

PROJETO

HEMOLACRIA

Curso: AMS – 1º Ano
Instituição: Fatec Ourinhos
Acadêmicos:
João Pedro Silva Machado de Oliveira

Cidade: Ourinhos
Ano: 2026

1. INTRODUÇÃO

Este documento tem como objetivo apresentar a documentação de requisitos e especificações do jogo **Hemolacria**, desenvolvido como projeto acadêmico do curso AMS – 1º Ano.

O projeto propõe o desenvolvimento de um Tower Defense com sistemas mecânicos de RPG e IDLE, utilizando sistemas de waves, progressão de personagem, estratégias de posicionamento dos personagens e construção de estruturas para a defesa do seu personagem contra os inimigos.

1.1 Tema

Desenvolvimento de um jogo digital do gênero Tower Defense, com foco em combate por waves e construção de estratégias utilizando os recursos gerados pelo próprio mapa e jogador.

1.2 Objetivo do Projeto

Criar um jogo singleplayer para PC em gênero Tower Defense e Construção de estruturas com o objetivo do projeto de:

- Sistema de Ondas ou Waves
- Combate interativo com upgrades
- Coleta de Recursos
- Construção de estruturas
- Estratégias com personagens e estruturas
- Diversidade de inimigos

O jogo será desenvolvido utilizando a engine Godot com linguagem GDScript, para design foi utilizado Aseprite e sendo atualmente documentado pelo Github:
Link: <https://github.com/JonesDevs/Hemolacria>

2. DESCRIÇÃO GERAL DO SISTEMA

2.1 Descrição do Problema

Towers Defense limitam muito a criatividade do jogador, simplificando sistemas e padronizando o jogo, removendo a profundidade estratégica e a criatividade.

O projeto **Hemolacria** tende a promover essas dificuldades, possibilitando ao jogador:

- Jogabilidade manual e automático
 - Gestão de Recursos
 - Sistema de ataque interativos
 - Possibilidade maior estrategicamente em posicionamento e defesa
-

2.2 Objetivo do Sistema

O sistema deverá:

- Apresentar a proposta do jogo desde o começo
 - Permitir o jogador a alternar entre jogar manualmente ou automático
 - Executar combates interativos
 - Permitir o manuseio de recursos e inventários
 - Possibilitar a melhoria de sua estrutura de defesa, a si mesmo e seus personagens
 - Conter estratégias de maneira criativa, sem nenhum meta
 - Desafiar o jogador, a manusear sua equipe, aliados e defesas.
-

2.3 Principais Características

- Plataforma: PC
- Modalidade: Singleplayer
- Gênero: Tower Defense
- Engine: Godot
- Linguagem: GDScript

2.4 Problemas e Limitações do Sistema

Durante o desenvolvimento do sistema podem surgir algumas limitações técnicas e de design que devem ser consideradas no planejamento do projeto.

PR_01 – Melhoria a longo prazo do Design

A qualidade visual do jogo, por ser independente, dificulta ter uma beleza visual grande.

PR_02 – Balanceamento Geral

A criação de um sistema equilibrado entre jogador e inimigos pode exigir ajustes constantes para evitar situações de combate muito fáceis ou excessivamente difíceis.

PR_03 – Inteligência Artificial Inimiga

A implementação de IA estratégica para inimigos pode apresentar dificuldades técnicas, especialmente na tomada de decisões.

PR_04 – Escalabilidade do Sistema

Com o aumento de habilidades, inimigos, estruturas e itens pode ocorrer aumento na complexidade de manutenção do código se a arquitetura não for planejada adequadamente.

PR_05 – Interface e Experiência do Usuário

Sistemas de Tower Defense com muitas mecânicas podem se tornar complexos para novos jogadores, exigindo cuidado no design da interface.

PR_06 – Tempo de Desenvolvimento

Por se tratar de um projeto acadêmico com prazo limitado, algumas funcionalidades podem precisar ser simplificadas ou implementadas em versões futuras.

3. REQUISITOS DO SISTEMA

3.1 Requisitos Funcionais

Sistema de Jogador

RF_07 – O sistema deve apresentar a possibilidade de upgrades diversos.

RF_08 – O sistema deve permitir o manuseamento manual e automático do jogador.

RF_09 – O sistema deve mostrar o sistema de atributos do jogador.

Sistema de Combate

RF_10 – O sistema deve permitir o jogador, a controlar suas tropas e manusear suas estruturas, podendo construí-las, destruí-las e posicioná-las.

- RF_11** – O sistema deve calcular os danos, curas, buff e debuffs de personagens e inimigos
- RF_12** – O sistema deve executar combate em Waves ou Ondas, definida por tempo e eliminações.
- RF_13** – O sistema deve permitir a opção de acelerar a Wave ou Ondas e também Pular ou Skippar para acelerar o Ciclo
- RF_14** – O sistema deve permitir uso de habilidades com custo de recurso (energia, sangue e etc.).
- RF_15** – O sistema deve aplicar efeitos de status (sangramento, envenenamento, maldição, congelamento e etc.).
- RF_16** – O sistema deve permitir uso de itens durante o combate.
- RF_17** – O sistema deve registrar a conclusão de cada Ciclo.
-

Sistema de Progressão

- RF_18** – O sistema deve permitir evoluir seu personagem e suas estruturas a custo de recursos ganhados pelos próprios inimigos e mapas.
- RF_19** – O sistema deve permitir a construção estrategicamente de melhorias a fim de melhorar a jogabilidade do jogador.
- RF_20** – O sistema mostrar a dificuldade após cada ciclo finalizado, deixando o jogo mais desafiador.
-

Sistema de Mundo

- RF_21** – O sistema deve permitir exploração de áreas.
- RF_22** – O sistema deve permitir interação com NPCs.
- RF_23** – O sistema deve permitir coleta de recursos ao explorar o mapa e após a finalização da wave, coletar os recursos dropados ou derrubados pelos os inimigos.
-

Sistema de Base

- RF_24** – O sistema deve permitir compra de itens com NPC's por custo de recursos.
- RF_25** – O sistema deve permitir troca de itens por outros recursos.
- RF_26** – O sistema deve permitir craft de equipamentos.
- RF_27** – O sistema deve permitir melhoria da base do jogador estrategicamente e criativamente.
-

Sistema de IA Inimiga

- RF_28** – O sistema deve controlar ações inimigas automaticamente.
- RF_29** – A IA deve utilizar ataques, buffs e debuffs estrategicamente.

Interface

RF_30 – O sistema deve exibir interface com barra de vida, recursos, Ciclos, Waves ou Ondas atuais.

RF_31 – O sistema deve apresentar inventário acessível ao jogador.

RF_32 – O sistema deve permitir navegação intuitiva entre menus.

3.2 Requisitos Não Funcionais

RNF_33 – O sistema deve armazenar dados localmente de forma segura e consistente.

RNF_34 – O código deve ser modular e organizado por camadas.

RNF_35 – A interface deve ser intuitiva e não exigir tutorial externo obrigatório.

RNF_36 – O sistema deve ser desenvolvido utilizando a engine Godot e Aseprite.

RNF_37 – O sistema deve permitir alternar entre manuseamento automático e manual do jogador.

RNF_3 – O sistema deve permitir opções de gráficos e vídeo para o jogador.

4. DIAGRAMAS

(ESPAÇO RESERVADO)

- Diagrama de Caso de Uso
 - Diagrama de Classes
 - Diagrama de Combate
 - Diagrama de Estrutura de Dados
-

5. PROTÓTIPOS DE TELAS

(ESPAÇO RESERVADO PARA INSERÇÃO FUTURA)

- Tela inicial
- Tela de seleção de personagem
- Tela de combate
- Tela de inventário
- Tela de loja

6. REGRAS DE NEGÓCIO

As regras de negócio definem como o sistema deve se comportar internamente para garantir coerência nas mecânicas do jogo.

6.1 Sistema de Atributos

RN_01 – Limite de Atributos

Cada personagem possuirá atributos base definidos no início do jogo, com valores mínimos e máximos estabelecidos pelo sistema.

RN_02 – Evolução de Atributos

Conforme o jogador evolui seu personagem, derrotando bosses ou inimigos, poderá transferir suas almas, em troca de pontos de atributos.

RN_03 – Influência dos Atributos

Os atributos devem influenciar diretamente nas mecânicas do jogo, como:

- Dano causado
 - Resistência a efeitos e Proteção
 - Iniciativa no combate
-

6.2 Sistema de Combate

RN_04 – Turnos de Combate

O combate seguirá um sistema de turnos baseado em **iniciativa**, onde personagens com maior valor de iniciativa agirão primeiro.

RN_05 – Rolagem de Dados

Todas as ações relevantes do combate (ataque, esquiva, habilidades e perícias) deverão utilizar um sistema de **rolagem de dados virtual** para determinar sucesso ou falha.

RN_06 – Uso de Recursos

Em alguns crafts utilizaram recursos como madeira, pedra e etc. As habilidades utilizaram apenas se precisarem.

RN_07 – Estados Alterados

Efeitos de status como sangramento, medo ou veneno, devem possuir duração limitada em turnos e aplicar penalidades ou danos progressivos.

6.3 Sistema de Progressão

RN_08 – Ganho de Experiência

O jogador deve receber experiência ao derrotar inimigos ou itens completando eventos específicos.

RN_09 – Evolução de Atributos

Ao atingir uma quantidade específica de experiência, o personagem sobe de nível e recebe melhorias como:

- Pontos de atributo
 - Aumento de recursos máximos.
-

6.4 Sistema de Itens e Inventário

RN_10 – Limite de Inventário

O inventário do jogador deverá possuir limite de espaço para armazenamento de itens.

RN_11 – Equipamentos

Itens de equipamento devem modificar atributos do personagem enquanto estiverem equipados.

RN_12 – Consumíveis

Itens consumíveis devem aplicar efeitos imediatos como cura, remoção de status ou aumento temporário de atributos.

6.5 Sistema de Loja

RN_13 – Economia do Jogo

Itens vendidos nas lojas devem possuir valores fixos ou baseados em raridade.

RN_14 – Troca de Recursos

Alguns itens poderão ser obtidos através de troca de materiais coletados durante a exploração.

RN_15 – Craft de Equipamentos

Equipamentos especiais poderão ser criados combinando materiais específicos coletados no jogo.